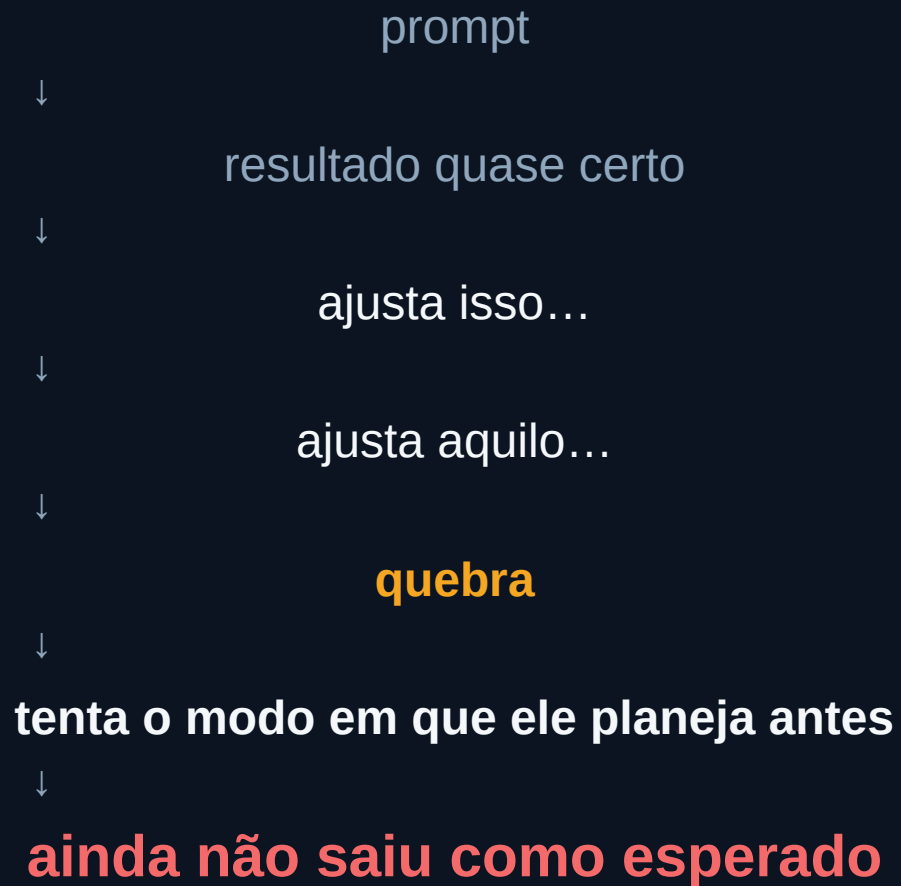


Confiança no código gerado por I.A

Do "ajusta isso, ajusta aquilo" ao processo que verifica por você.

Essa cena é familiar?



Os 5 níveis de adoção de I.A

L0	Hater	Sem I.A no fluxo
L1	Copy/Paste	Cola do chat, autocomplete ocasional
L2	Babá	Aprova cada passo; passa o tempo esperando
L3	Gerente	Aprova plano, revisa o PR final
L4	Fábrica	Pipelines autônomos pra fluxos repetíveis

Digita no chat: **em que nível você está?**

Código escrito por I.A $\neq$ produtividade

Especialmente quando a maior parte do tempo é gasta parado, vendo a I.A escrever.

O resultado diz mais sobre **como você usa** do que sobre o que a I.A consegue fazer.

Como **umentar a confiança** no resultado
sem **umentar a supervisão**?

Um LLM não sabe que seu repo existe

Um **gerador de texto** que prevê a continuação mais provável do que recebeu.

Sem estado

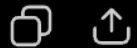
Sem memória

Sem acesso

A prova

existe um bug no cálculo de desconto, corrija

Claro. Envie o trecho do código responsável pelo cálculo do desconto (e, se possível, um exemplo do valor esperado vs. o valor atual) que eu corrijo o bug.



Sem acesso ao repo, resta pedir: *"envie o trecho do código que eu corrijo o bug."*

Harness

O software **ao redor** do modelo.

Está para o modelo como o **runner de CI** está para o seu script:
dá acesso ao repo, executa comandos, devolve o resultado.

Harness + Modelo = Agente de código

Mas acesso não basta

Pedido vago → o agente decide pelo **dado de treinamento**, não pelas **suas convenções**.

Contexto ≈ o que cabe na mesa do revisor.

A gaveta inteira, ele se perde; a pasta certa, ele acerta.

Guias — antecipam

Direcionam o agente antes de agir:

Regras e padrões de código

Estrutura de pastas

Blueprints de arquitetura

Anti-padrões ("nunca faça X")

Aumentam o acerto **na primeira tentativa.**

Sensores — verificam

Observam o resultado depois que o agente age:

Testes automatizados

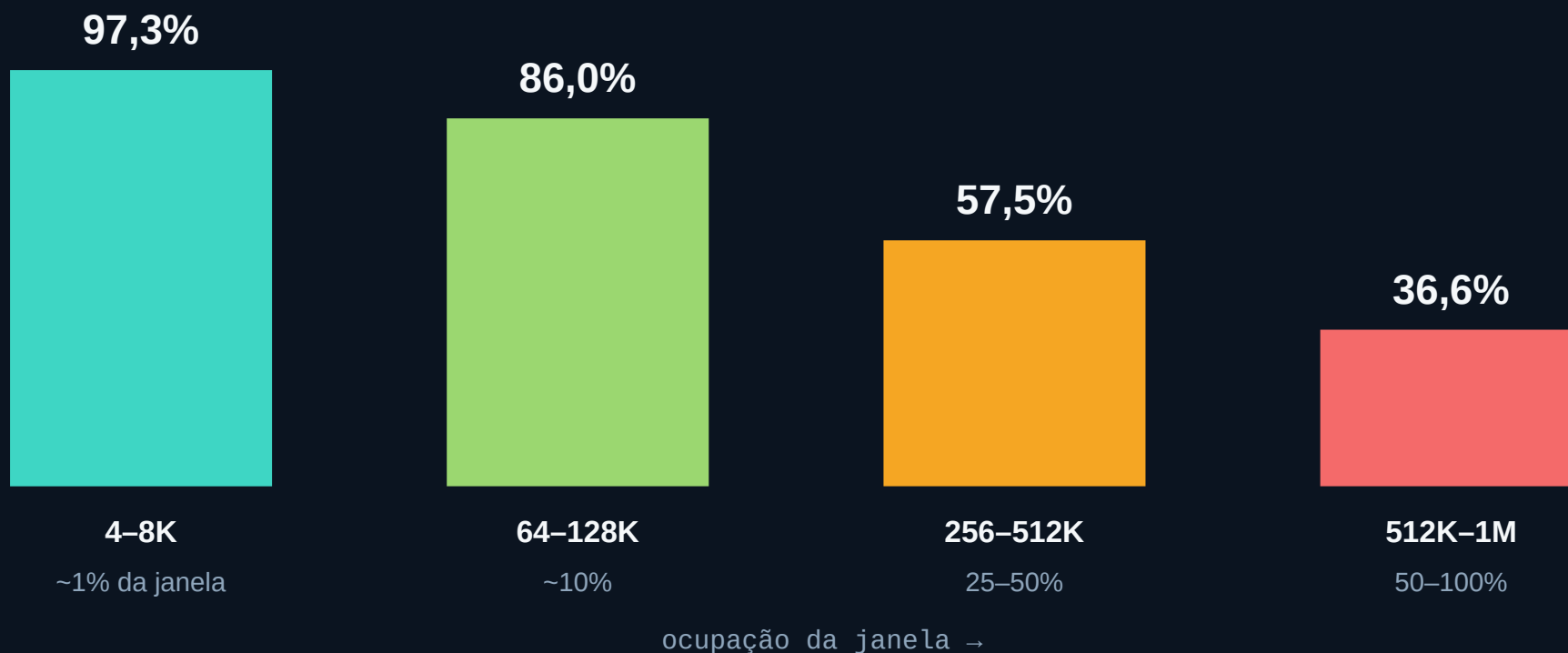
Lintar, compilador, typecheck

Browser, logs, banco

Sensores são o seu **CI apontado pro agente.**

Quanto mais cheia a janela, pior o resultado

Janela de contexto = tudo que o modelo considera de uma vez — regras, logs de teste, código, conversa.



Spec-Driven Development

Em vez de *prompt* → *código* → *conserta depois*:

spec → **plano** → **tasks** → **executar** → **verificar**

— cada etapa gera um artefato aprovado antes da próxima.

Isso não é processo novo.

É o PRD → spec → tasks → review que vocês já usam entre humanos — aplicado ao agente.

Você aprova o **quê** e o plano — os sensores vigiam o **como**.

Cada fase entrega um artefato aprovável



Azul = você aprova o artefato · **Cinza** = sensores verificam

Como isso responde cada problema

Chute

Guias que você aprovou

Janela degrada

Uma fase por vez, reset entre fases

Erro silencioso

Sensores por fase: review, QA E2E, regressão

Auto-review viciado

Review em contexto isolado — outro revisor, de mesa limpa, só o diff e a spec

Cada fase é uma **skill** $\approx$ runbook de deploy que executa sozinho, carregado sob demanda.

Requisito revisável antes do código

tasks/01-painel-clima/prd.md

Objetivos

- Clima atual de uma cidade em até 2 interações
- 100% das chamadas a provedores externos passam pelo backend
- Dados visíveis em até ~2s após a seleção

Requisitos funcionais

2. Busca com mais de uma localidade → apresentar opções para desambiguar
3. Nenhuma localidade → "cidade não encontrada" sem quebrar a interface

A decisão técnica registrada, com trade-off

tasks/01-painel-clima/techspec.md

Resumo executivo

Padrão BFF: o frontend fala só com o backend;
o backend é o único a consumir o Open-Meteo.

Principais decisões

- Sem cache server-side: simplicidade; mitiga rate-limit via debounce no frontend.
Trade-off: maior exposição a latência – aceitável no escopo atual, revisável sem mudar o contrato.

O plano vira checklist executável

tasks/01-painel-clima/01_task.md

- [x] 1.0 Backend BFF (busca e clima)
 - [x] 1.1 zod + config.ts (URLs, timeout, faixas lat/lon)
 - [x] 1.2 lib/http.ts (AbortController/timeout)
 - + weather-code.ts (WMO → pt-BR)
 - [x] 1.4 open-meteo.service.ts (normalização + isCurrent)
 - [x] 1.7 testes unitários e de integração
 - (fetch mockado · cenários 200 / 400 / 502)

O review achou um problema de verdade

tasks/01-painel-clima/codereview.md

Status: APROVADO COM RESSALVAS

Testes: 80/80 passando (74 unit/integração + 6 E2E)

Coverage: 96,4% statements

Severidade	Arquivo	Descrição
Média	lib/api.ts	BFF_BASE_URL hardcoded – quebrará em qualquer deploy

Isso vocês já sabem julgar. Auditar artefato é mais barato que auditar linha.

A confiança vem do processo, não do modelo

Chute → **Guias**

Erro silencioso → **Sensores**

Janela degradada → **Uma fase por vez**

Supervisão infinita → **Artefato auditável**

O mesmo motivo pelo qual você confia no **CI**, não na memória do dev.

r-spec — o SDD que vamos rodar ao vivo

Minha implementação do padrão: uma skill por fase do pipeline, instalável em Claude Code, Codex, Cursor e outros.

terminal

```
npx skills add https://github.com/renanloureiroo/r-spec
```

Próxima sessão: do PRD ao PR, ao vivo, com uma feature real.

Tragam uma feature candidata do backlog de vocês.

Pros L2 e L3 do chat: essa sessão é pra vocês.